1. Introduction

Après avoir défini la vision globale, le périmètre du Produit Minimum Viable (MVP), l'architecture technique et l'organisation des fichiers du backend de Quantara, cette étape est dédiée à la **conception détaillée de la base de données**. Il est crucial de définir avec précision les entités à persister, leurs relations et leurs attributs pour construire une fondation robuste et évolutive.

L'objectif est de traduire les besoins fonctionnels et architecturaux en un modèle de données concret, qui servira de base pour l'implémentation des modèles SQLAlchemy, des repositories et des services métier. Une conception de base de données réfléchie dès le départ est essentielle pour garantir la performance, l'intégrité des données et la facilité de maintenance du système.

2. Objectif de l'Étape

Le but de cette étape est de concevoir une base de données qui soit :

- **Cohérente avec le MVP :** Contenant uniquement les entités et les attributs strictement nécessaires pour les fonctionnalités du MVP.
- **Alignée avec les Modules Backend :** Chaque entité doit clairement correspondre à un domaine métier défini dans l'architecture modulaire.
- **Simple et Rapide à Implémenter :** Évitant la sur-ingénierie pour permettre un développement agile.

- **Propre et Extensible :** Conçue selon les bonnes pratiques pour faciliter les évolutions futures sans refonte majeure.

Nous cherchons à trouver un équilibre entre une base de données trop minimaliste qui bloquerait rapidement le produit, et une base trop complexe qui ralentirait le développement initial. L'objectif est une **base minimale, propre, relationnelle et extensible**.

3. Principe Général de la Base de Données Quantara

La base de données PostgreSQL de Quantara sera le référentiel de vérité pour toutes les informations critiques et persistantes liées à l'utilisateur et au fonctionnement interne de la plateforme. Elle doit stocker :

- Les données propres à l'utilisateur (profil, portefeuille, progression).
- Les informations nécessaires au suivi du produit et à son fonctionnement.
- Les données persistantes dans le temps et indépendantes des fournisseurs externes.

En revanche, la base de données ne doit pas forcément stocker massivement toutes les données de marché externes, les bougies historiques complètes ou les flux temps réel. Ces données peuvent être récupérées dynamiquement via des fournisseurs externes et mises en cache si nécessaire, sans encombrer la base de données principale.

Séparation des Données :

Données à Persister (MVP)	Données à Ne Pas Forcément Persister (MVP)
Utilisateurs	Données de marché en temps réel
Portefeuille	Candles historiques complètes
Positions	Snapshots marché volumineux
Trades	Indicateurs calculés à la volée
Cours	
Leçons	
Progression des leçons	

4. Les Grandes Entités du MVP

Basées sur l'architecture et les modules définis précédemment, les entités principales et obligatoires pour le MVP sont :

- **User** (Module `identity`)
- **Portfolio** (Module `simulation`)
- **Position** (Module `simulation`)
- **Trade** (Module `simulation`)
- **Course** (Module `education`)
- **Lesson** (Module `education`)
- **LessonProgress** (Module `progress`)

Ces entités représentent les tables fondamentales nécessaires au fonctionnement du MVP.

5. Vue d'Ensemble des Relations MVP

Un schéma conceptuel simplifié des relations entre les entités clés est le suivant :

```
User
 ├── Portfolio (1:1)
 │      ├── Position (1:N)
 │      └── Trade (1:N)
 │
 └── LessonProgress (1:N)

Course
 └── Lesson (1:N)

LessonProgress (N:1 avec User, N:1 avec Lesson)
```

Cette vue fournit une base claire et lisible, suffisante pour démarrer l'implémentation.

6. Description Détaillée des Tables MVP

6.1 Table `users`

Rôle : Stocker les informations de base des utilisateurs de la plateforme.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique de l'utilisateur	Clé primaire
`first_name`	String	Prénom de l'utilisateur	
`last_name`	String	Nom de famille de l'utilisateur	
`email`	String	Adresse email de l'utilisateur	Unique, Obligatoire
`hashed_password`	String	Mot de passe haché de l'utilisateur	Obligatoire
`is_active`	Boolean	Statut d'activation du compte	Défaut: `true`
`created_at`	Timestamp	Date et heure de création du compte	
`updated_at`	Timestamp	Date et heure de dernière mise à jour	

Centralité : Presque toutes les autres entités du système dépendent d'un utilisateur.

6.2 Table `portfolios`

Rôle : Représenter le portefeuille virtuel principal d'un utilisateur.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique du portefeuille	Clé primaire
`user_id`	UUID/Int	Référence à l'utilisateur propriétaire	Clé étrangère (`users.id`)
`cash_balance`	Numeric	Solde de trésorerie disponible dans le portefeuille	Défaut: `0.0`
`created_at`	Timestamp	Date et heure de création	
`updated_at`	Timestamp	Date et heure de dernière mise à jour	

Relation : Un portefeuille appartient à un utilisateur. Au MVP, un utilisateur aura un seul portefeuille principal. Des portefeuilles multiples (ex: paper trading) pourront être

ajoutés ultérieurement.

6.3 Table `positions`

Rôle : Représenter les positions ouvertes ou fermées sur des actifs au sein d'un portefeuille.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique de la position	Clé primaire
`portfolio_id`	UUID/Int	Référence au portefeuille parent	Clé étrangère (`portfolios.id`)
`symbol`	String	Symbole de l'actif (ex: BTC, ETH)	Obligatoire
`quantity`	Numeric	Quantité de l'actif détenue	Obligatoire
`average_entry_price`	Numeric	Prix moyen d'acquisition de la position	Obligatoire
`status`	String	Statut de la position (ex: `open` , `closed`)	Obligatoire
`created_at`	Timestamp	Date et heure de création	
`updated_at`	Timestamp	Date et heure de dernière mise à jour	

Utilité : Cette table permet de connaître l'exposition actuelle de l'utilisateur sans avoir à recalculer toutes les transactions historiques.

6.4 Table `trades`

Rôle : Stocker l'historique détaillé de toutes les opérations de trading simulées.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique du trade	Clé primaire
`portfolio_id`	UUID/Int	Référence au portefeuille concerné	Clé étrangère (`portfolios.id`)
`symbol`	String	Symbole de l'actif tradé	Obligatoire
`side`	String	Type d'opération (ex: `BUY`, `SELL`)	Obligatoire
`amount`	Numeric	Montant total de l'opération	Obligatoire
`quantity`	Numeric	Quantité d'actif échangée	Obligatoire
`price`	Numeric	Prix unitaire de l'actif au moment du trade	Obligatoire
`stop_loss`	Numeric	Prix de stop-loss (optionnel)	Nullable
`take_profit`	Numeric	Prix de take-profit (optionnel)	Nullable
`created_at`	Timestamp	Date et heure de l'exécution du trade	

Essentielle : C'est la source de vérité pour l'historique de simulation de l'utilisateur. Au MVP, cette table peut suffire pour l'historique des transactions, sans nécessiter une table `transaction_history` séparée.

6.5 Table `courses`

Rôle : Stocker les informations sur les cours éducatifs disponibles sur la plateforme.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique du cours	Clé primaire
`title`	String	Titre du cours	Obligatoire
`description`	Text	Description détaillée du cours	
`level`	String	Niveau de difficulté (ex: `beginner`, `intermediate`)	
`created_at`	Timestamp	Date et heure de création	

Remarque MVP : Simple et suffisant pour la gestion des cours.

6.6 Table `lessons`

Rôle : Stocker les leçons individuelles qui composent chaque cours.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique de la leçon	Clé primaire
`course_id`	UUID/Int	Référence au cours parent	Clé étrangère (`courses.id`)
`title`	String	Titre de la leçon	Obligatoire
`content`	Text	Contenu de la leçon (Markdown, HTML, etc.)	Obligatoire
`order_index`	Integer	Ordre d'affichage de la leçon dans le cours	Obligatoire
`created_at`	Timestamp	Date et heure de création	

Utilité `order_index` : Permet de maintenir un ordre pédagogique clair pour les leçons.

6.7 Table `lesson_progress`

Rôle : Stocker la progression de chaque utilisateur pour chaque leçon.

Champ	Type	Description	Contraintes
`id`	UUID/Int	Identifiant unique de l'entrée de progression	Clé primaire
`user_id`	UUID/Int	Référence à l'utilisateur	Clé étrangère (`users.id`)
`lesson_id`	UUID/Int	Référence à la leçon	Clé étrangère (`lessons.id`)
`completed`	Boolean	Indique si la leçon est complétée	Défaut: `false`
`completed_at`	Timestamp	Date et heure de complétion de la leçon	Nullable

Utilité MVP : Cette table est suffisante pour suivre la progression. La progression globale d'un cours peut être calculée dynamiquement à partir du nombre de leçons complétées par rapport au nombre total de leçons du cours. Une table `course_progress` séparée n'est pas nécessaire au MVP.

7. Tables Optionnelles (Reportées au-delà du MVP)

Certaines tables peuvent être envisagées pour des phases ultérieures, mais ne sont pas obligatoires pour le MVP afin de maintenir la simplicité et la rapidité de développement.

- **`course_progress`** : Non nécessaire si la progression est calculable dynamiquement depuis `lesson_progress`.
- **`market_snapshots`** : Non nécessaire au début, les données de marché peuvent être récupérées à la volée.

- `settings` : Les préférences utilisateur peuvent être fusionnées dans la table `users` ou gérées de manière simplifiée au MVP.
- `insights` : Non nécessaire au MVP si les insights sont calculés à la volée ou si le module est minimal.

8. Contraintes et Index Recommandés

Même au stade du MVP, l'ajout de contraintes et d'index est crucial pour l'intégrité des données et les performances des requêtes.

Table	Contraintes / Index Recommandés
`users`	`UNIQUE(email)`
`portfolios`	`FOREIGN KEY (user_id) REFERENCES users(id)`
`positions`	`FOREIGN KEY (portfolio_id) REFERENCES portfolios(id)`, `INDEX(portfolio_id)`, `INDEX(symbol)`
`trades`	`FOREIGN KEY (portfolio_id) REFERENCES portfolios(id)`, `INDEX(portfolio_id)`, `INDEX(created_at)`
`courses`	
`lessons`	`FOREIGN KEY (course_id) REFERENCES courses(id)`, `INDEX(course_id)`, `INDEX(order_index)`
`lesson_progress`	`FOREIGN KEY (user_id) REFERENCES users(id)`, `FOREIGN KEY (lesson_id) REFERENCES lessons(id)`, `INDEX(user_id)`, `INDEX(lesson_id)`, `UNIQUE(user_id, lesson_id)`

La contrainte d'unicité sur `(user_id, lesson_id)` dans `lesson_progress` est particulièrement importante pour éviter les doublons de progression.

9. Champs d'État et Timestamps

Certains champs sont essentiels pour la gestion de l'état et le suivi temporel des entités :

- **`users.is_active`** : Pour activer ou désactiver un compte utilisateur.
- **`positions.status`** : Pour indiquer si une position est `open` ou `closed`.
- **`lesson_progress.completed`** : Pour simplifier les requêtes de progression.

Timestamps : Toutes les tables principales devraient inclure au minimum `created_at` et `updated_at`. Pour la table `trades`, `created_at` est souvent suffisant. Pour `lesson_progress`, `completed_at` est un champ utile pour le suivi précis.

10. Choix de Modélisation Simplifiés pour le MVP

Pour ne pas sur-complexifier la base de données au démarrage, les choix de modélisation suivants sont adoptés :

- **Un seul portefeuille par utilisateur :** Simplifie la gestion du portefeuille initial.
- **Pas de table séparée pour l'historique des transactions :** La table `trades` est suffisante pour le MVP.
- **Pas de table `course_progress` :** La progression est calculée dynamiquement à partir de `lesson_progress`.
- **Pas de persistance massive des données de marché :** Les données sont récupérées à la volée via les fournisseurs externes.

Ces choix permettent une implémentation plus rapide et plus ciblée pour le MVP.

11. Ce qu'il Faut Absolument Éviter dans le Design DB

Pour préserver la propreté et l'efficacité de la base de données, les pratiques suivantes doivent être évitées :

- **Créer des tables "au cas où" :** Chaque table doit avoir une justification claire et immédiate pour le MVP.
- **Stocker des données de fournisseur inutiles :** Ne pas dupliquer des données externes qui peuvent être récupérées à la demande.
- **Dupliquer l'état entre plusieurs tables sans besoin :** Maintenir une source de vérité unique pour chaque information.
- **Concevoir une base de données pour 3 ans dès le MVP :** Se concentrer sur les besoins actuels et anticiper l'extensibilité, sans sur-ingénierie.
- **Mélanger des domaines métier distincts dans une seule table :** Respecter la séparation des préoccupations (ex: ne pas mélanger contenu éducatif, simulation et authentification dans une même table).

12. Base de Données MVP Finale Recommandée

La base de données recommandée pour le MVP de Quantara est composée des tables suivantes :

```
users
portfolios
positions
trades
courses
lessons
lesson_progress
```

Cette base est :

- **Minimale :** Contenant l'essentiel pour le MVP.
- **Cohérente :** Alignée avec l'architecture modulaire.
- **Suffisante :** Répondant aux besoins fonctionnels immédiats.
- **Extensible :** Conçue pour faciliter les ajouts futurs.

13. Conclusion

Cette étape a permis de définir une conception de base de données claire et structurée pour le backend de Quantara. En détaillant les entités, leurs relations, leurs attributs et les choix de modélisation pour le MVP, nous avons posé les fondations d'un système de persistance de données robuste et performant.

Cette spécification est désormais prête à être traduite en modèles SQLAlchemy et à guider l'implémentation des repositories et des services. Elle garantit que la base de données sera un atout pour le développement du MVP, et non une source de complexité ou de dette technique.